

Lecture 2: Binary-Coded Genetic Algorithm (BGA)

Course: Evolutionary Algorithms

Instructor: Engr. Muhammad Farooq

Learning outcomes

By the end of this lecture, students should be able to:

- explain what a binary-coded genetic algorithm is

Learning outcomes

By the end of this lecture, students should be able to:

- explain what a binary-coded genetic algorithm is
- define basic GA terminology

Learning outcomes

By the end of this lecture, students should be able to:

- explain what a binary-coded genetic algorithm is
- define basic GA terminology
- explain the generalized EC framework

Learning outcomes

By the end of this lecture, students should be able to:

- explain what a binary-coded genetic algorithm is
- define basic GA terminology
- explain the generalized EC framework
- encode real decision variables using binary strings

Learning outcomes

By the end of this lecture, students should be able to:

- explain what a binary-coded genetic algorithm is
- define basic GA terminology
- explain the generalized EC framework
- encode real decision variables using binary strings
- decode binary strings into actual variable values

Learning outcomes

By the end of this lecture, students should be able to:

- explain what a binary-coded genetic algorithm is
- define basic GA terminology
- explain the generalized EC framework
- encode real decision variables using binary strings
- decode binary strings into actual variable values
- evaluate a population for a minimization problem

Learning outcomes

By the end of this lecture, students should be able to:

- explain what a binary-coded genetic algorithm is
- define basic GA terminology
- explain the generalized EC framework
- encode real decision variables using binary strings
- decode binary strings into actual variable values
- evaluate a population for a minimization problem
- explain parent selection, crossover, mutation, and survivor selection

Learning outcomes

By the end of this lecture, students should be able to:

- explain what a binary-coded genetic algorithm is
- define basic GA terminology
- explain the generalized EC framework
- encode real decision variables using binary strings
- decode binary strings into actual variable values
- evaluate a population for a minimization problem
- explain parent selection, crossover, mutation, and survivor selection
- understand one full generation of BGA through a worked example

Learning outcomes

By the end of this lecture, students should be able to:

- explain what a binary-coded genetic algorithm is
- define basic GA terminology
- explain the generalized EC framework
- encode real decision variables using binary strings
- decode binary strings into actual variable values
- evaluate a population for a minimization problem
- explain parent selection, crossover, mutation, and survivor selection
- understand one full generation of BGA through a worked example
- run a simple Python demonstration of BGA

Basic terminology

Core terms

- **Chromosome:** a full encoded solution

More terms

Basic terminology

Core terms

- **Chromosome:** a full encoded solution
- **Gene:** one position in the chromosome

More terms

Basic terminology

Core terms

- **Chromosome:** a full encoded solution
- **Gene:** one position in the chromosome
- **Allele:** value a gene can take, usually 0 or 1 in BGA

More terms

Core terms

- **Chromosome:** a full encoded solution
- **Gene:** one position in the chromosome
- **Allele:** value a gene can take, usually 0 or 1 in BGA
- **Individual:** one candidate solution

More terms

Core terms

- **Chromosome:** a full encoded solution
- **Gene:** one position in the chromosome
- **Allele:** value a gene can take, usually 0 or 1 in BGA
- **Individual:** one candidate solution
- **Population:** a set of candidate solutions

More terms

Basic terminology

Core terms

- **Chromosome:** a full encoded solution
- **Gene:** one position in the chromosome
- **Allele:** value a gene can take, usually 0 or 1 in BGA
- **Individual:** one candidate solution
- **Population:** a set of candidate solutions

More terms

- **Fitness / objective value:** quality of a solution

Example chromosome: 10110101

Basic terminology

Core terms

- **Chromosome:** a full encoded solution
- **Gene:** one position in the chromosome
- **Allele:** value a gene can take, usually 0 or 1 in BGA
- **Individual:** one candidate solution
- **Population:** a set of candidate solutions

More terms

- **Fitness / objective value:** quality of a solution
- **Parent:** selected for reproduction

Example chromosome: 10110101

Basic terminology

Core terms

- **Chromosome:** a full encoded solution
- **Gene:** one position in the chromosome
- **Allele:** value a gene can take, usually 0 or 1 in BGA
- **Individual:** one candidate solution
- **Population:** a set of candidate solutions

More terms

- **Fitness / objective value:** quality of a solution
- **Parent:** selected for reproduction
- **Offspring:** created from parents

Example chromosome: 10110101

Basic terminology

Core terms

- **Chromosome:** a full encoded solution
- **Gene:** one position in the chromosome
- **Allele:** value a gene can take, usually 0 or 1 in BGA
- **Individual:** one candidate solution
- **Population:** a set of candidate solutions

More terms

- **Fitness / objective value:** quality of a solution
- **Parent:** selected for reproduction
- **Offspring:** created from parents
- **Generation:** one full cycle of the algorithm

Example chromosome: 10110101

Basic terminology

Core terms

- **Chromosome:** a full encoded solution
- **Gene:** one position in the chromosome
- **Allele:** value a gene can take, usually 0 or 1 in BGA
- **Individual:** one candidate solution
- **Population:** a set of candidate solutions

More terms

- **Fitness / objective value:** quality of a solution
- **Parent:** selected for reproduction
- **Offspring:** created from parents
- **Generation:** one full cycle of the algorithm
- **Mating pool:** selected parents used for crossover

Example chromosome: 10110101

Basic terminology

Core terms

- **Chromosome:** a full encoded solution
- **Gene:** one position in the chromosome
- **Allele:** value a gene can take, usually 0 or 1 in BGA
- **Individual:** one candidate solution
- **Population:** a set of candidate solutions

More terms

- **Fitness / objective value:** quality of a solution
- **Parent:** selected for reproduction
- **Offspring:** created from parents
- **Generation:** one full cycle of the algorithm
- **Mating pool:** selected parents used for crossover
- **Survivor selection:** choosing the next generation

Example chromosome: 10110101

What is Binary-Coded Genetic Algorithm?

A **Binary-Coded Genetic Algorithm (BGA)** is a genetic algorithm in which each solution is represented by a string of bits.

Example chromosome:

0110011010

What is Binary-Coded Genetic Algorithm?

A **Binary-Coded Genetic Algorithm (BGA)** is a genetic algorithm in which each solution is represented by a string of bits.

Example chromosome:

0110011010

If the problem has multiple variables, the chromosome is divided into segments:

01100 11010

What is Binary-Coded Genetic Algorithm?

A **Binary-Coded Genetic Algorithm (BGA)** is a genetic algorithm in which each solution is represented by a string of bits.

Example chromosome:

0110011010

If the problem has multiple variables, the chromosome is divided into segments:

01100 11010

- first 5 bits may represent x_1
- next 5 bits may represent x_2

What is Binary-Coded Genetic Algorithm?

A **Binary-Coded Genetic Algorithm (BGA)** is a genetic algorithm in which each solution is represented by a string of bits.

Example chromosome:

0110011010

If the problem has multiple variables, the chromosome is divided into segments:

01100 11010

- first 5 bits may represent x_1
- next 5 bits may represent x_2

Important point: the original optimization problem may be continuous, but the GA representation is binary.

Generalized framework of EC techniques

- 1 Choose solution representation

Generalized framework of EC techniques

- ① Choose solution representation
- ② Initialize generation counter

Generalized framework of EC techniques

- 1 Choose solution representation
- 2 Initialize generation counter
- 3 Generate initial random population $P(t)$

Generalized framework of EC techniques

- ① Choose solution representation
- ② Initialize generation counter
- ③ Generate initial random population $P(t)$
- ④ Evaluate $P(t)$

Generalized framework of EC techniques

- ① Choose solution representation
- ② Initialize generation counter
- ③ Generate initial random population $P(t)$
- ④ Evaluate $P(t)$
- ⑤ While termination condition is not satisfied:
 - select parents from $P(t)$ to form mating pool $M(t)$
 - apply variation to $M(t)$ to create offspring $Q(t)$
 - evaluate $Q(t)$
 - choose survivors from $P(t)$ and $Q(t)$ to form $P(t + 1)$
 - increase generation counter

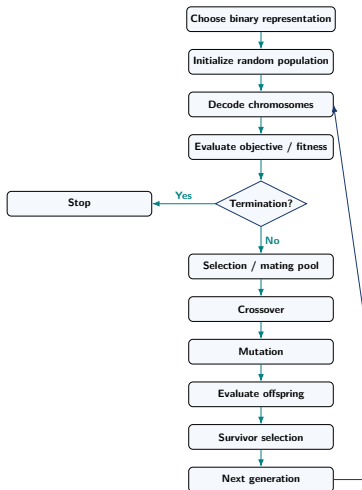
Generalized framework of EC techniques

- ① Choose solution representation
- ② Initialize generation counter
- ③ Generate initial random population $P(t)$
- ④ Evaluate $P(t)$
- ⑤ While termination condition is not satisfied:
 - select parents from $P(t)$ to form mating pool $M(t)$
 - apply variation to $M(t)$ to create offspring $Q(t)$
 - evaluate $Q(t)$
 - choose survivors from $P(t)$ and $Q(t)$ to form $P(t + 1)$
 - increase generation counter

Meaning of symbols:

$P(t)$: parent population, $M(t)$: mating pool, $Q(t)$: offspring population, $P(t+1)$: next generation

Flowchart of Binary-Coded GA



Why binary coding?

- it is easy to understand

Why binary coding?

- it is easy to understand
- mutation is simple as bit flipping

Why binary coding?

- it is easy to understand
- mutation is simple as bit flipping
- crossover is easy to visualize

Why binary coding?

- it is easy to understand
- mutation is simple as bit flipping
- crossover is easy to visualize
- it can represent bounded real variables through encoding

Why binary coding?

- it is easy to understand
- mutation is simple as bit flipping
- crossover is easy to visualize
- it can represent bounded real variables through encoding
- it is a very good first step for learning genetic algorithms

Why binary coding?

- it is easy to understand
- mutation is simple as bit flipping
- crossover is easy to visualize
- it can represent bounded real variables through encoding
- it is a very good first step for learning genetic algorithms

Limitation: precision depends on string length.

Binary representation of a real variable

Suppose a real variable x_i has:

- lower bound $x_i^{(L)}$

Binary representation of a real variable

Suppose a real variable x_i has:

- lower bound $x_i^{(L)}$
- upper bound $x_i^{(U)}$

Binary representation of a real variable

Suppose a real variable x_i has:

- lower bound $x_i^{(L)}$
- upper bound $x_i^{(U)}$
- binary substring length l

Binary representation of a real variable

Suppose a real variable x_i has:

- lower bound $x_i^{(L)}$
- upper bound $x_i^{(U)}$
- binary substring length l

First decode the binary substring into an integer $DV(s)$, then map it to the actual range.

Binary representation of a real variable

Suppose a real variable x_i has:

- lower bound $x_i^{(L)}$
- upper bound $x_i^{(U)}$
- binary substring length l

First decode the binary substring into an integer $DV(s)$, then map it to the actual range.

$$x_i = x_i^{(L)} + \frac{x_i^{(U)} - x_i^{(L)}}{2^l - 1} DV(s)$$

Binary representation of a real variable

Suppose a real variable x_i has:

- lower bound $x_i^{(L)}$
- upper bound $x_i^{(U)}$
- binary substring length l

First decode the binary substring into an integer $DV(s)$, then map it to the actual range.

$$x_i = x_i^{(L)} + \frac{x_i^{(U)} - x_i^{(L)}}{2^l - 1} DV(s)$$

Here, $DV(s)$ is the decoded integer value of the binary substring.

Example of decoding

- $l = 5$

Example of decoding

- $l = 5$
- lower bound $x_i^{(L)} = 3$

Example of decoding

- $l = 5$
- lower bound $x_i^{(L)} = 3$
- upper bound $x_i^{(U)} = 10$

Example of decoding

- $l = 5$
- lower bound $x_i^{(L)} = 3$
- upper bound $x_i^{(U)} = 10$

Then:

$$x_i = 3 + \frac{7}{31}DV(s)$$

Example of decoding

- $l = 5$
- lower bound $x_i^{(L)} = 3$
- upper bound $x_i^{(U)} = 10$

Then:

$$x_i = 3 + \frac{7}{31}DV(s)$$

If the binary string is

11011

Example of decoding

- $l = 5$
- lower bound $x_i^{(L)} = 3$
- upper bound $x_i^{(U)} = 10$

Then:

$$x_i = 3 + \frac{7}{31}DV(s)$$

If the binary string is

11011

then its decoded value is:

$$DV(s) = 1 \cdot 2^4 + 1 \cdot 2^3 + 0 \cdot 2^2 + 1 \cdot 2^1 + 1 \cdot 2^0 = 27$$

Example of decoding

- $l = 5$
- lower bound $x_i^{(L)} = 3$
- upper bound $x_i^{(U)} = 10$

Then:

$$x_i = 3 + \frac{7}{31}DV(s)$$

If the binary string is

11011

then its decoded value is:

$$DV(s) = 1 \cdot 2^4 + 1 \cdot 2^3 + 0 \cdot 2^2 + 1 \cdot 2^1 + 1 \cdot 2^0 = 27$$

Therefore:

$$x_i = 3 + \frac{7}{31}(27) = 9.096$$

Precision of binary representation

$$\Delta = \frac{x_i^{(U)} - x_i^{(L)}}{2^l - 1}$$

This means:

- binary coding approximates a continuous variable

Precision of binary representation

$$\Delta = \frac{x_i^{(U)} - x_i^{(L)}}{2^l - 1}$$

This means:

- binary coding approximates a continuous variable
- more bits give better precision

Precision of binary representation

$$\Delta = \frac{x_i^{(U)} - x_i^{(L)}}{2^l - 1}$$

This means:

- binary coding approximates a continuous variable
- more bits give better precision
- fewer bits give coarser representation

Worked optimization example

We solve the Rosenbrock minimization problem:

$$f(x_1, x_2) = 100(x_2 - x_1^2)^2 + (1 - x_1)^2$$

subject to:

$$-5 \leq x_1 \leq 5, \quad -4 \leq x_2 \leq 4$$

Worked optimization example

We solve the Rosenbrock minimization problem:

$$f(x_1, x_2) = 100(x_2 - x_1^2)^2 + (1 - x_1)^2$$

subject to:

$$-5 \leq x_1 \leq 5, \quad -4 \leq x_2 \leq 4$$

The optimum is:

$$x^* = (1, 1)^T, \quad f(x^*) = 0$$

Worked optimization example

We solve the Rosenbrock minimization problem:

$$f(x_1, x_2) = 100(x_2 - x_1^2)^2 + (1 - x_1)^2$$

subject to:

$$-5 \leq x_1 \leq 5, \quad -4 \leq x_2 \leq 4$$

The optimum is:

$$x^* = (1, 1)^T, \quad f(x^*) = 0$$

Why this example?

- the problem is continuous
- the search space is bounded
- it shows how BGA solves continuous problems through encoding

Chromosome structure for this example

Let total chromosome length be 10 bits.

- first 5 bits represent x_1

Chromosome structure for this example

Let total chromosome length be 10 bits.

- first 5 bits represent x_1
- next 5 bits represent x_2

Chromosome structure for this example

Let total chromosome length be 10 bits.

- first 5 bits represent x_1
- next 5 bits represent x_2

So one chromosome looks like:

01100 11010

Chromosome structure for this example

Let total chromosome length be 10 bits.

- first 5 bits represent x_1
- next 5 bits represent x_2

So one chromosome looks like:

01100 11010

Population size: $N = 8$

Initial random population

Index	Chromosome	$DV(x_1)$	$DV(x_2)$
1	01100 11010	12	26
2	11000 01011	24	11
3	00110 00110	6	6
4	01000 10111	8	23
5	10100 11101	20	29
6	01101 01000	13	8
7	00101 11011	5	27
8	11100 11000	28	24

Decoding formulas for the example

For x_1 :

$$x_1 = -5 + \frac{10}{31}DV(x_1)$$

For x_2 :

$$x_2 = -4 + \frac{8}{31}DV(x_2)$$

Idea: each 5-bit substring is first converted to an integer, then mapped to its allowed real interval.

Decoded population

Index	Chromosome	x_1	x_2
1	01100 11010	-1.129	2.710
2	11000 01011	2.742	-1.161
3	00110 00110	-3.065	-2.452
4	01000 10111	-2.419	1.935
5	10100 11101	1.452	3.484
6	01101 01000	-0.806	-1.935
7	00101 11011	-3.387	2.968
8	11100 11000	4.032	2.194

Evaluate the population

Index	Chromosome	x_1	x_2	$f(x_1, x_2)$
1	01100 11010	-1.129	2.710	210.445
2	11000 01011	2.742	-1.161	7536.407
3	00110 00110	-3.065	-2.452	14041.882
4	01000 10111	-2.419	1.935	1546.603
5	10100 11101	1.452	3.484	189.732
6	01101 01000	-0.806	-1.935	671.924
7	00101 11011	-3.387	2.968	7252.209
8	11100 11000	4.032	2.194	19793.183

Since this is a **minimization** problem, lower objective value means better solution.

Termination condition

The algorithm may stop if:

- generation count exceeds the maximum allowed

Termination condition

The algorithm may stop if:

- generation count exceeds the maximum allowed
- a sufficiently good objective value is found

Termination condition

The algorithm may stop if:

- generation count exceeds the maximum allowed
- a sufficiently good objective value is found
- there is no meaningful improvement

Termination condition

The algorithm may stop if:

- generation count exceeds the maximum allowed
- a sufficiently good objective value is found
- there is no meaningful improvement

At this stage, since we are only at the initial generation, the algorithm continues.

Selection is used to identify promising solutions for reproduction.

- **Parent selection:** chooses who will reproduce

Selection is used to identify promising solutions for reproduction.

- **Parent selection:** chooses who will reproduce
- **Survivor selection:** chooses who remains in the next generation

Selection is used to identify promising solutions for reproduction.

- **Parent selection:** chooses who will reproduce
- **Survivor selection:** chooses who remains in the next generation
- In this lecture, parent selection is done by **binary tournament**

Selection is used to identify promising solutions for reproduction.

- **Parent selection:** chooses who will reproduce
- **Survivor selection:** chooses who remains in the next generation
- In this lecture, parent selection is done by **binary tournament**

Main idea:

- good solutions should have a higher chance of being selected
- weaker solutions are not removed immediately
- this keeps a balance between progress and diversity

Binary tournament selection (part 1)

Rule:

- choose two solutions randomly

Binary tournament selection (part 1)

Rule:

- choose two solutions randomly
- compare their objective values

Binary tournament selection (part 1)

Rule:

- choose two solutions randomly
- compare their objective values
- since this is a minimization problem, the lower objective value wins

Binary tournament selection (part 1)

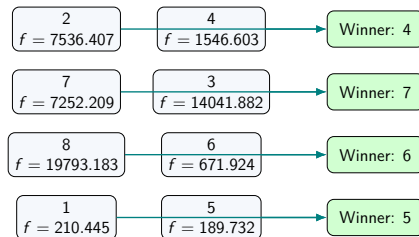
Rule:

- choose two solutions randomly
- compare their objective values
- since this is a minimization problem, the lower objective value wins
- repeat until the mating pool is full

Binary tournament selection (part 1)

- Rule:
- choose two solutions randomly
 - compare their objective values
 - since this is a minimization problem, the lower objective value wins
 - repeat until the mating pool is full

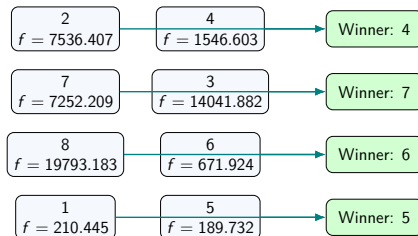
First four tournament selections



Binary tournament selection (part 1)

- Rule:
- choose two solutions randomly
 - compare their objective values
 - since this is a minimization problem, the lower objective value wins
 - repeat until the mating pool is full

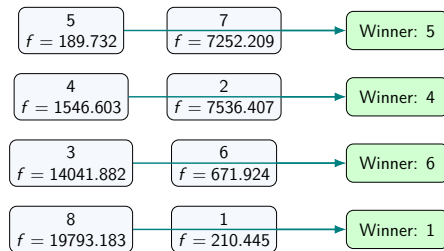
First four tournament selections



Winners so far: 4, 7, 6, 5

Binary tournament selection (part 2)

Next four tournament selections



Winners in this part: 5, 4, 6, 1

Complete mating-pool list: 4, 7, 6, 5, 5, 4, 6, 1

Tournament winners and mating pool

So the tournament winners are:

4, 7, 6, 5, 5, 4, 6, 1

These selected winners form the **mating pool**. Notice that some indices appear more than once because stronger solutions can win multiple tournaments.

Mating pool

New index	Chromosome	x_1	x_2	$f(x_1, x_2)$
1	01000 10111	-2.419	1.935	1546.603
2	00101 11011	-3.387	2.968	7252.209
3	01101 01000	-0.806	-1.935	671.924
4	10100 11101	1.452	3.484	189.732
5	10100 11101	1.452	3.484	189.732
6	01000 10111	-2.419	1.935	1546.603
7	01101 01000	-0.806	-1.935	671.924
8	01100 11010	-1.129	2.710	210.445

Pair them as:

$\{4, 7\}$, $\{8, 2\}$, $\{5, 1\}$, $\{6, 3\}$

Crossover operator

Crossover creates new solutions by exchanging parts of two parent chromosomes.

It is performed with crossover probability:

$$p_c = 0.9$$

Crossover operator

Crossover creates new solutions by exchanging parts of two parent chromosomes.

It is performed with crossover probability:

$$p_c = 0.9$$

Types of crossover:

- single-point crossover
- n -point crossover
- uniform crossover

Crossover operator

Crossover creates new solutions by exchanging parts of two parent chromosomes.

It is performed with crossover probability:

$$p_c = 0.9$$

Types of crossover:

- single-point crossover
- n -point crossover
- uniform crossover

In this lecture, we use **single-point crossover**.

Crossover: Pair 1 {4, 7}

Let random number be:

$$r = 0.75$$

Since $0.75 < 0.9$, perform crossover at the **8th bit**.

Parents

1010011101 0110101000

Offspring

1010011100 0110101001

Interpretation: the left segment stays from the original parent, while the right segment is swapped after the crossover point.

Decoded results: $O4 : f = 125.336$, $O7 : f = 545.121$

Crossover: Pair 2 {8, 2}

Let random number be:

$$r = 0.23$$

Since $0.23 < 0.9$, perform crossover at the **3rd bit**.

Parents

0110011010 0010111011

Offspring

0110111011 0010011010

One offspring improved, while the other became much worse. This shows that crossover recombines material, but does not guarantee better children every time.

Decoded results: $O8 : f = 540.287$, $O2 : f = 12236.916$

Crossover: Pair 3 and Pair 4

Pair 3: {5, 1}

If $r = 0.93 > 0.9$, no crossover happens, so the parents are copied directly.

Pair 4: {6, 3}

If $r = 0.68 < 0.9$, perform crossover at the **6th bit**.

Parents

0100010111 0110101000

Offspring

0100011000 0110100111

Decoded results: $O6 : f = 1351.054$, $O3 : f = 812.047$

Offspring population after crossover

Index	Chromosome	x_1	x_2	$f(x_1, x_2)$
O4	1010011100	1.452	3.226	125.336
O7	0110101001	-0.806	-1.677	545.121
O8	0110111011	-0.806	2.968	540.287
O2	0010011010	-3.710	2.710	12236.916
O5	1010011101	1.452	3.484	189.732
O1	0100010111	-2.419	1.935	1546.603
O6	0100011000	-2.419	2.194	1351.054
O3	0110100111	-0.806	-2.194	812.047

Interpretation of crossover

Crossover can produce:

- better solutions

Interpretation of crossover

Crossover can produce:

- better solutions
- worse solutions

Interpretation of crossover

Crossover can produce:

- better solutions
- worse solutions
- nearly unchanged solutions

Interpretation of crossover

Crossover can produce:

- better solutions
- worse solutions
- nearly unchanged solutions

Important point: crossover does not guarantee improvement in every child, but over generations it helps explore the search space.

Mutation operator

Mutation:

- maintains diversity

Mutation operator

Mutation:

- maintains diversity
- introduces new bit patterns

Mutation operator

Mutation:

- maintains diversity
- introduces new bit patterns
- helps avoid stagnation

Mutation operator

Mutation:

- maintains diversity
- introduces new bit patterns
- helps avoid stagnation

In BGA, mutation is usually implemented as bit flipping.

Mutation operator

Mutation:

- maintains diversity
- introduces new bit patterns
- helps avoid stagnation

In BGA, mutation is usually implemented as bit flipping.

Let:

$$p_m = 0.10$$

Mutation operator

Mutation:

- maintains diversity
- introduces new bit patterns
- helps avoid stagnation

In BGA, mutation is usually implemented as bit flipping.

Let:

$$p_m = 0.10$$

Mutation happens when:

$$r < p_m$$

Mutation examples

Suppose mutation random numbers are:

Index	r
1	0.05
2	0.32
3	0.15
4	0.01
5	0.06
6	0.24
7	0.50
8	0.54

Since $p_m = 0.10$, mutation occurs for solutions 1, 4, 5. Now we inspect exactly which bit changed in each case.

Mutation in detail: solutions 1 and 4

Solution 1

0100010111 $\rightarrow$ 0101010111

Changed bit: **4th bit**, from 0 $\rightarrow$ 1

Before: $f = 1546.603$ After: $f = 154.658$

Solution 4

1010011100 $\rightarrow$ 1010111100

Changed bit: **5th bit**, from 0 $\rightarrow$ 1

Before: $f = 125.336$ After: $f = 1.208$

Mutation in detail: solution 5 and interpretation

Solution 5

1010011101 $\rightarrow$ 0010011101

Changed bit: **1st bit**, from 1 $\rightarrow$ 0

Before: $f = 189.732$ After: $f = 10585.571$

Important point: mutation can improve a solution or damage it. Its role is not to improve every child, but to maintain diversity and introduce new patterns that crossover alone may not create.

Offspring population after mutation

Index	Chromosome	x_1	x_2	$f(x_1, x_2)$
1	0101010111	-1.774	1.935	154.658
2	0010011010	-3.710	2.710	12236.916
3	0110100111	-0.806	-2.194	812.047
4	1010111100	1.774	3.226	1.208
5	0010011101	-3.710	3.484	10585.571
6	0100011000	-2.419	2.194	1351.054
7	0110101001	-0.806	-1.677	545.121
8	0110111011	-0.806	2.968	540.287

Survivor selection

After creating offspring, we must select the next generation.
This stage is called:

- survivor selection

After creating offspring, we must select the next generation.

This stage is called:

- survivor selection
- elimination

Survivor selection

After creating offspring, we must select the next generation.

This stage is called:

- survivor selection
- elimination
- environmental selection

After creating offspring, we must select the next generation.

This stage is called:

- survivor selection
- elimination
- environmental selection

Use the $(\mu + \lambda)$ -strategy:

- μ : parent population
- λ : offspring population

Survivor selection

After creating offspring, we must select the next generation.

This stage is called:

- survivor selection
- elimination
- environmental selection

Use the $(\mu + \lambda)$ -strategy:

- μ : parent population
- λ : offspring population

Combine both populations and choose the best N solutions. Since this is minimization, choose the smallest objective values.

Best survivors

Choose the best 8 solutions from parents and offspring:

O4, O1, P5, P1, O8, O7, P6, O3

These selected survivors form the next generation.

Next generation population

Index	Chromosome	x_1	x_2	$f(x_1, x_2)$
1	1010111100	1.774	3.226	1.208
2	0101010111	-1.774	1.935	154.658
3	10100 11101	1.452	3.484	189.732
4	01100 11010	-1.129	2.710	210.445
5	0110111011	-0.806	2.968	540.287
6	0110101001	-0.806	-1.677	545.121
7	01101 01000	-0.806	-1.935	671.924
8	0110100111	-0.806	-2.194	812.047

Main observations from the worked example

- binary strings can represent real-valued decision variables

Main observations from the worked example

- binary strings can represent real-valued decision variables
- selection gives good solutions more reproductive opportunities

Main observations from the worked example

- binary strings can represent real-valued decision variables
- selection gives good solutions more reproductive opportunities
- crossover may improve or damage a solution

Main observations from the worked example

- binary strings can represent real-valued decision variables
- selection gives good solutions more reproductive opportunities
- crossover may improve or damage a solution
- mutation may improve or damage a solution

Main observations from the worked example

- binary strings can represent real-valued decision variables
- selection gives good solutions more reproductive opportunities
- crossover may improve or damage a solution
- mutation may improve or damage a solution
- survivor selection prevents poor offspring from dominating

Main observations from the worked example

- binary strings can represent real-valued decision variables
- selection gives good solutions more reproductive opportunities
- crossover may improve or damage a solution
- mutation may improve or damage a solution
- survivor selection prevents poor offspring from dominating
- improvement happens statistically over generations

This code uses:

- deap

This code uses:

- `deap`
- `numpy`

This code uses:

- `deap`
- `numpy`
- `matplotlib`

Python demonstration

This code uses:

- `deap`
- `numpy`
- `matplotlib`

It demonstrates:

- binary chromosome
- decoding to real variables
- Rosenbrock minimization
- binary tournament selection
- one-point crossover
- bit-flip mutation

Python code (1/4)

```
1 import random
2 import numpy as np
3 import matplotlib.pyplot as plt
4 from deap import base, creator, tools, algorithms
5
6 BITS_PER_VAR = 5
7 NUM_VARS = 2
8 CHROM_LENGTH = BITS_PER_VAR * NUM_VARS
9
10 X1_L, X1_U = -5, 5
11 X2_L, X2_U = -4, 4
12
13 POP_SIZE = 8
14 NGEN = 20
15 CXPB = 0.9
16 MUTPB = 0.1
17 BIT_MUTPB = 1.0 / CHROM_LENGTH
18
19 random.seed(42)
20 np.random.seed(42)
```

Python code (2/4)

```
1 def bits_to_int(bits):
2     return int("".join(str(b) for b in bits), 2)
3
4 def decode_variable(bits, lower, upper):
5     dv = bits_to_int(bits)
6     max_dv = (2 ** len(bits)) - 1
7     value = lower + ((upper - lower) / max_dv) * dv
8     return dv, value
9
10 def decode_individual(individual):
11     x1_bits = individual[:BITS_PER_VAR]
12     x2_bits = individual[BITS_PER_VAR:]
13     dv1, x1 = decode_variable(x1_bits, X1_L, X1_U)
14     dv2, x2 = decode_variable(x2_bits, X2_L, X2_U)
15     return dv1, dv2, x1, x2
16
17 def rosenbrock(individual):
18     _, _, x1, x2 = decode_individual(individual)
19     f = 100 * (x2 - x1**2) ** 2 + (1 - x1) ** 2
20     return (f,)
```

Python code (3/4)

```
1 if not hasattr(creator, "FitnessMinBGA"):
2     creator.create("FitnessMinBGA", base.Fitness, weights=(-1.0,))
3 if not hasattr(creator, "IndividualBGA"):
4     creator.create("IndividualBGA", list, fitness=creator.FitnessMinBGA)
5
6 toolbox = base.Toolbox()
7 toolbox.register("attr_bool", random.randint, 0, 1)
8 toolbox.register("individual", tools.initRepeat,
9                 creator.IndividualBGA, toolbox.attr_bool, CHROM_LENGTH)
10 toolbox.register("population", tools.initRepeat, list, toolbox.individual)
11
12 toolbox.register("evaluate", rosenbrock)
13 toolbox.register("select", tools.selTournament, tournsize=2)
14 toolbox.register("mate", tools.cxOnePoint)
15 toolbox.register("mutate", tools.mutFlipBit, indpb=BIT_MUTPB)
```

Python code (4/5)

```
1 def main():
2     population = toolbox.population(n=POP_SIZE)
3     stats = tools.Statistics(lambda ind: ind.fitness.values[0])
4     stats.register("min", np.min)
5     stats.register("avg", np.mean)
6     hof = tools.HallOfFame(1)
7
8     population, logbook = algorithms.eaMuPlusLambda(
9     population, toolbox,
10    mu=POP_SIZE, lambda_=POP_SIZE,
11    cxpb=CXPB, mutpb=MUTPB,
12    ngen=NGEN, stats=stats, halloffame=hof, verbose=True
13    )
14
15    best = hof[0]
16    dv1, dv2, x1, x2 = decode_individual(best)
17    best_f = best.fitness.values[0]
18    print("Best chromosome:", "".join(map(str, best)))
19    print("DV(x1), DV(x2):", dv1, dv2)
20    print("Decoded x1, x2:", round(x1, 3), round(x2, 3))
21    print("Best objective:", round(best_f, 6))
22    generations = logbook.select("gen")
```

Python code (5/5)

```
1 min_vals = logbook.select("min")
2 avg_vals = logbook.select("avg")
3
4 plt.figure(figsize=(10, 6))
5 plt.plot(generations, min_vals, marker="o", label="Best objective")
6 plt.plot(generations, avg_vals, marker="s", label="Average objective")
7 plt.xlabel("Generation")
8 plt.ylabel("Objective value")
9 plt.title("Binary-Coded GA on Rosenbrock Function")
10 plt.legend()
11 plt.grid(True)
12 plt.show()
13
14 if __name__ == "__main__":
15     main()
```

What the Python demo shows

- first 5 bits represent x_1

What the Python demo shows

- first 5 bits represent x_1
- next 5 bits represent x_2

What the Python demo shows

- first 5 bits represent x_1
- next 5 bits represent x_2
- each chromosome is decoded into real values

What the Python demo shows

- first 5 bits represent x_1
- next 5 bits represent x_2
- each chromosome is decoded into real values
- Rosenbrock objective is minimized

What the Python demo shows

- first 5 bits represent x_1
- next 5 bits represent x_2
- each chromosome is decoded into real values
- Rosenbrock objective is minimized
- one-point crossover recombines parents

What the Python demo shows

- first 5 bits represent x_1
- next 5 bits represent x_2
- each chromosome is decoded into real values
- Rosenbrock objective is minimized
- one-point crossover recombines parents
- bit-flip mutation introduces diversity

What the Python demo shows

- first 5 bits represent x_1
- next 5 bits represent x_2
- each chromosome is decoded into real values
- Rosenbrock objective is minimized
- one-point crossover recombines parents
- bit-flip mutation introduces diversity
- better solutions tend to appear over generations

Discussion questions

- Why do we need decoding in binary-coded GA?

Discussion questions

- Why do we need decoding in binary-coded GA?
- Why does bit length affect precision?

Discussion questions

- Why do we need decoding in binary-coded GA?
- Why does bit length affect precision?
- Why can crossover create worse offspring?

Discussion questions

- Why do we need decoding in binary-coded GA?
- Why does bit length affect precision?
- Why can crossover create worse offspring?
- Why is mutation probability usually small?

Discussion questions

- Why do we need decoding in binary-coded GA?
- Why does bit length affect precision?
- Why can crossover create worse offspring?
- Why is mutation probability usually small?
- What is the difference between parent selection and survivor selection?

Discussion questions

- Why do we need decoding in binary-coded GA?
- Why does bit length affect precision?
- Why can crossover create worse offspring?
- Why is mutation probability usually small?
- What is the difference between parent selection and survivor selection?
- Why is survivor selection useful?

Discussion questions

- Why do we need decoding in binary-coded GA?
- Why does bit length affect precision?
- Why can crossover create worse offspring?
- Why is mutation probability usually small?
- What is the difference between parent selection and survivor selection?
- Why is survivor selection useful?

Expected answers

- Decoding maps bit strings to actual variable values
- More bits allow finer precision
- Crossover recombines parts without guaranteeing improvement
- Mutation should be small to preserve useful structure
- Parent selection chooses who reproduces
- Survivor selection decides who remains

Quick quiz

1. In BGA, a real variable is handled by:

- A. ignoring its bounds
- B. converting it into a binary string and decoding it
- C. replacing it with a fixed constant
- D. using no objective function

Answer: B

2. In binary tournament selection for minimization, the winner is:

- A. the larger objective value
- B. the smaller objective value
- C. always the first one
- D. always the second one

Answer: B

End-of-lecture summary

Students should leave this lecture with these key ideas:

- BGA uses binary strings to encode solutions

End-of-lecture summary

Students should leave this lecture with these key ideas:

- BGA uses binary strings to encode solutions
- even continuous problems can be solved through binary encoding

End-of-lecture summary

Students should leave this lecture with these key ideas:

- BGA uses binary strings to encode solutions
- even continuous problems can be solved through binary encoding
- each variable is decoded from a bit substring

End-of-lecture summary

Students should leave this lecture with these key ideas:

- BGA uses binary strings to encode solutions
- even continuous problems can be solved through binary encoding
- each variable is decoded from a bit substring
- the BGA cycle is:

End-of-lecture summary

Students should leave this lecture with these key ideas:

- BGA uses binary strings to encode solutions
- even continuous problems can be solved through binary encoding
- each variable is decoded from a bit substring
- the BGA cycle is:
 - initialize

End-of-lecture summary

Students should leave this lecture with these key ideas:

- BGA uses binary strings to encode solutions
- even continuous problems can be solved through binary encoding
- each variable is decoded from a bit substring
- the BGA cycle is:
 - initialize
 - decode

End-of-lecture summary

Students should leave this lecture with these key ideas:

- BGA uses binary strings to encode solutions
- even continuous problems can be solved through binary encoding
- each variable is decoded from a bit substring
- the BGA cycle is:
 - initialize
 - decode
 - evaluate

End-of-lecture summary

Students should leave this lecture with these key ideas:

- BGA uses binary strings to encode solutions
- even continuous problems can be solved through binary encoding
- each variable is decoded from a bit substring
- the BGA cycle is:
 - initialize
 - decode
 - evaluate
 - select parents

End-of-lecture summary

Students should leave this lecture with these key ideas:

- BGA uses binary strings to encode solutions
- even continuous problems can be solved through binary encoding
- each variable is decoded from a bit substring
- the BGA cycle is:
 - initialize
 - decode
 - evaluate
 - select parents
 - crossover

End-of-lecture summary

Students should leave this lecture with these key ideas:

- BGA uses binary strings to encode solutions
- even continuous problems can be solved through binary encoding
- each variable is decoded from a bit substring
- the BGA cycle is:
 - initialize
 - decode
 - evaluate
 - select parents
 - crossover
 - mutation

End-of-lecture summary

Students should leave this lecture with these key ideas:

- BGA uses binary strings to encode solutions
- even continuous problems can be solved through binary encoding
- each variable is decoded from a bit substring
- the BGA cycle is:
 - initialize
 - decode
 - evaluate
 - select parents
 - crossover
 - mutation
 - evaluate offspring

End-of-lecture summary

Students should leave this lecture with these key ideas:

- BGA uses binary strings to encode solutions
- even continuous problems can be solved through binary encoding
- each variable is decoded from a bit substring
- the BGA cycle is:
 - initialize
 - decode
 - evaluate
 - select parents
 - crossover
 - mutation
 - evaluate offspring
 - survivor selection

End-of-lecture summary

Students should leave this lecture with these key ideas:

- BGA uses binary strings to encode solutions
- even continuous problems can be solved through binary encoding
- each variable is decoded from a bit substring
- the BGA cycle is:
 - initialize
 - decode
 - evaluate
 - select parents
 - crossover
 - mutation
 - evaluate offspring
 - survivor selection
 - next generation

End-of-lecture summary

Students should leave this lecture with these key ideas:

- BGA uses binary strings to encode solutions
- even continuous problems can be solved through binary encoding
- each variable is decoded from a bit substring
- the BGA cycle is:
 - initialize
 - decode
 - evaluate
 - select parents
 - crossover
 - mutation
 - evaluate offspring
 - survivor selection
 - next generation
- improvement happens statistically over generations